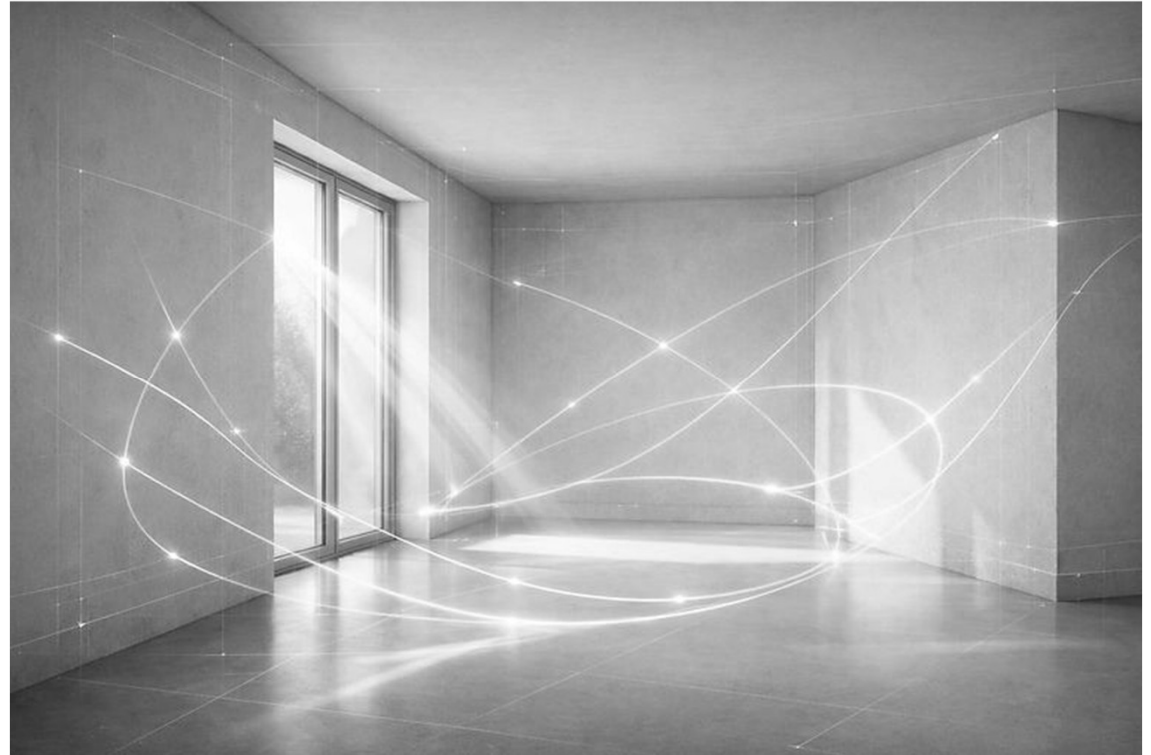


Lecture 6 • Global Illumination and Shadows

- *Module focus:* path tracing, photon mapping, shadow mapping, and shadow volumes.
- *Goal today:* move from “lighting looks wrong” to “I can choose and implement a shadow or GI method.”



Lecturer: Van-Tien Giap, Ph.D
Faculty of Engineering and Technology

Learning Outcomes and Session Map

- Explain how global illumination differs from local shading, and why shadows are a visibility problem.
- Describe the logic behind path tracing, photon mapping, and shadow volumes.
- Improve design choices by matching technique to quality, speed, and hardware constraints.
- Process graphics with programming tools by implementing step-by-step shader or ray-based routines.

1. Light transport

direct vs
indirect
rendering
equation

2. Core methods

path
tracing
photon
mapping

3. Real- time shadows

shadow
mapping
shadow
volumes

Why This Topic Matters in Computer Graphics

- Global illumination adds **believable** energy transfer: colored bounce light, corner darkening, and soft ambient lift.
- Shadows create **spatial relationships**: contact, scale, depth ordering, and time-of-day cues.
- The same scene may need different methods for *offline realism*, *real-time* interaction, or *limited hardware*.
- Good engineers choose the *cheapest method* that still preserves the visual message of the design.

Visual realism

bounce light and shadows make forms read as physical objects, not flat cutouts

Spatial cue

contact shadows and occlusion tell the eye where objects sit in space

Engineering trade-off

every method balances quality, speed, memory, and implementation effort

Design decision

choose the cheapest technique that preserves the visual story of the scene

Lighting is where visual design and rendering engineering meet.

Direct Lighting vs. Global Illumination

- **Direct lighting:** paths go from a light source >> surface >> the eye after **one interaction**.
- **Indirect lighting:** one or **more** additional **bounces** from other surfaces before reaching the eye.
- **Global illumination** is the combined solution: **direct + indirect** transport across the full scene.



Indirect only

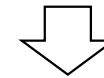
Soft energy transfer without sharp source lighting.

Direct only

Fast, crisp, but misses bounce light.

Full GI

Most believable because both components are present.



ceilings and corners often look unnaturally dark and disconnected.

Light Transport Intuition

- Outgoing radiance = emitted light + reflected incoming light.
- Reflected light depends on material response (BRDF), visibility, angle, and distance.
- The full problem is global because any surface may illuminate any other surface after enough bounces.
- Naively considering every interaction is expensive, so rendering uses *sampling*, *caching*, or *precomputation*.

1. Incoming light

where did the light arrive from?

2. Surface response

how does the material reflect it?

3. Visibility

is the path blocked or clear?

4. Geometry

what do angle and distance change?

5. Observation

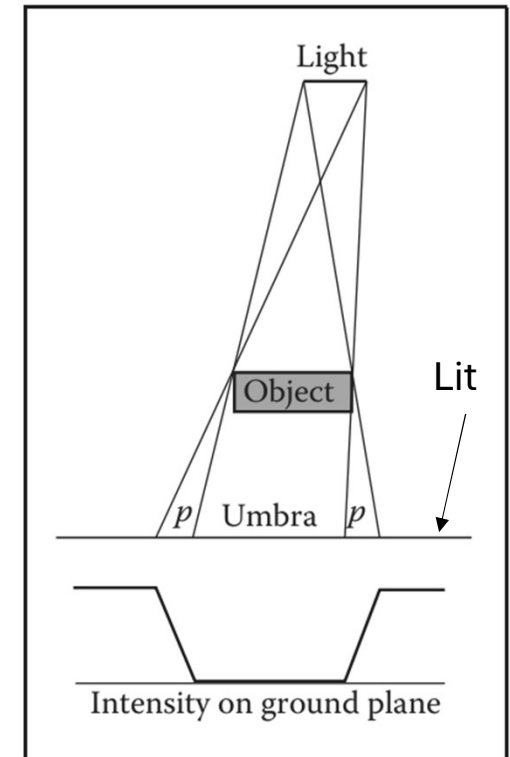
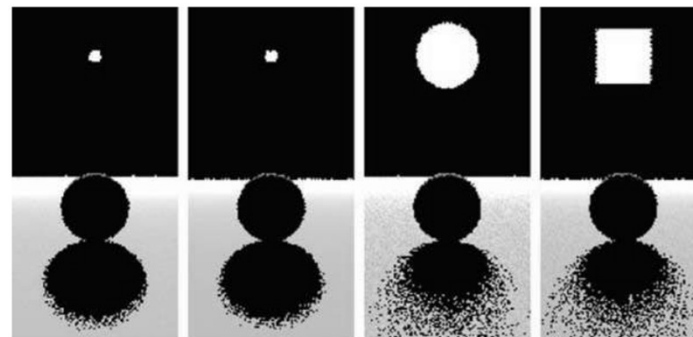
what radiance finally leaves the surface toward the eye?

Every rendering method stores or approximates some part of this transport chain.

Shadow Families: Hard, Soft, Image-Space, Geometry-Space

- **Hard shadows:** a light is treated as a point or when visibility is binary.
- **Soft shadows** contain a **penumbra** because only part of an area light is visible from a point.
- Image-space methods store visibility in **textures** or depth maps; geometry-space methods build shadow regions explicitly.
- Today we connect soft-shadow intuition to two practical families: shadow mapping and shadow volumes.

Soft shadows of different light sources



Umbra

0% of the light source is visible.

Penumbra

Some but not all light samples are visible.

Lit region

The full light source is visible.

Particle tracing

Radiance of a Surface:

$$L = \frac{\Phi_e + R\Phi_i}{\pi A}$$

Φ_e = emitted power,

Φ_i = incident power,

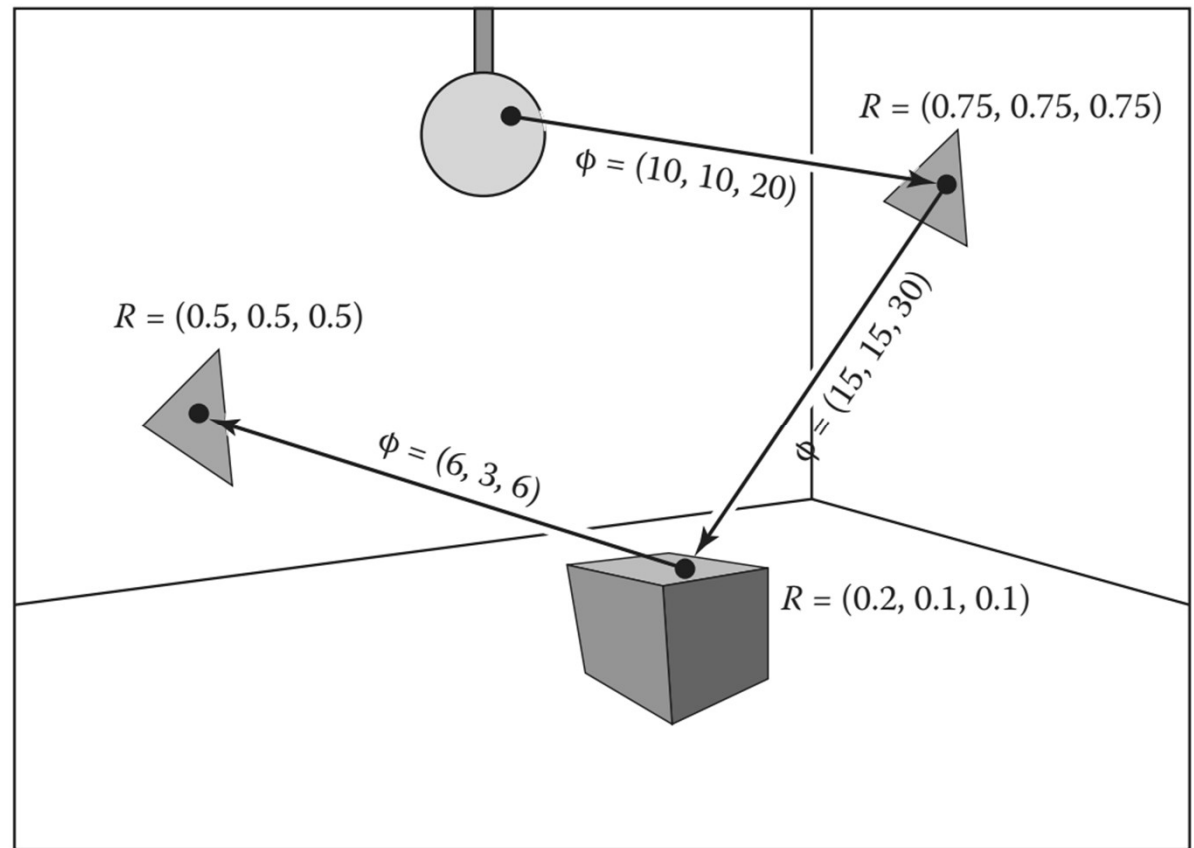
R = reflectance

A = area

+ to calculate incident power
from source i to face j

$$\Phi_{i \rightarrow j} = \pi L_i A_i F_{i \rightarrow j}$$

=> Too expensive, suitable for
offline rendering



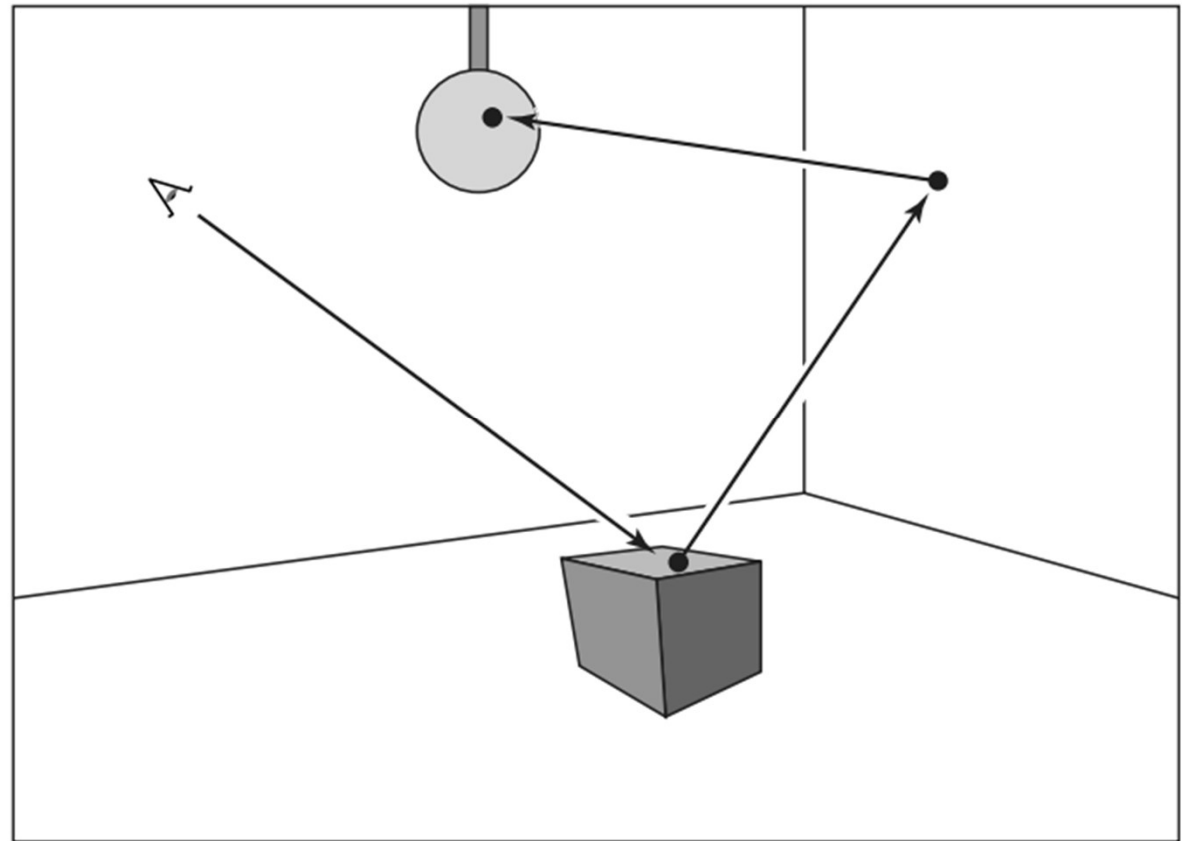
Path tracing

Sends number of random rays from the eye and traces them back to the light and then average all the light source they hit

$$\int_{x \in S} g(x) d\mu \approx \frac{1}{N} \sum_{i=1}^N \frac{g(x_i)}{p(x_i)}$$

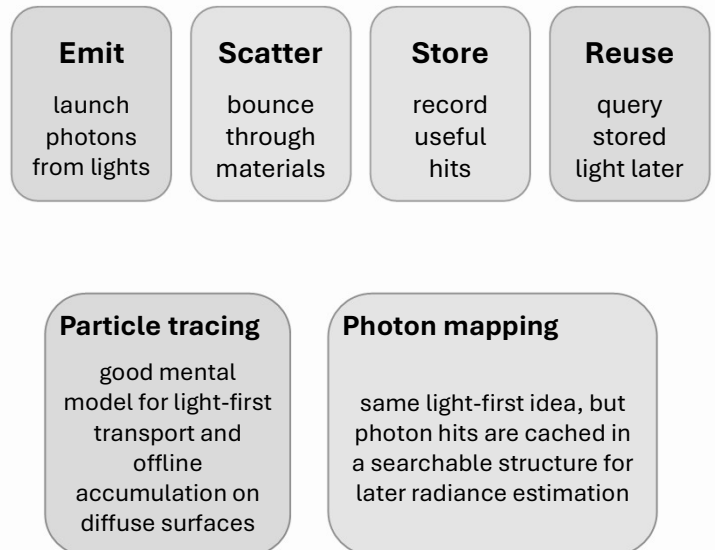
$g(x_i)$ = total light contribution of light source i to current pixel

$p(x_i)$ = probability density of algorithm choosing direction i ($\sim$ number of time choosing that direction)



From Particle Tracing to Photon Mapping

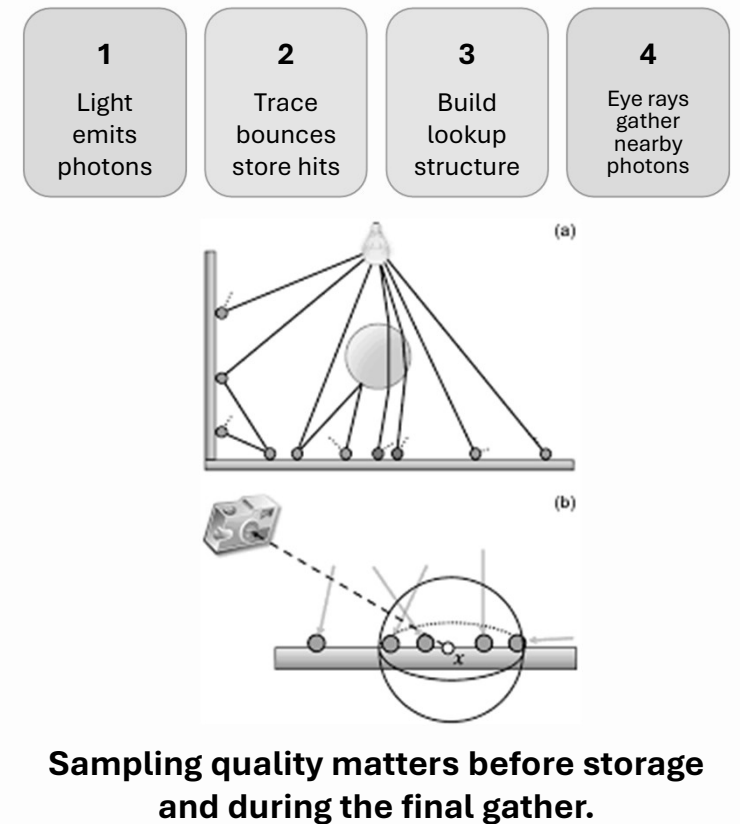
- Particle tracing sends light particles from luminaires into the scene and records where they land.
- This is useful for diffuse indirect lighting because many surface texels can accumulate incoming energy offline.
- **Photon mapping** extends the same idea by **storing** photon hits in a spatial data structure for later radiance estimates.
- The key design change is separating “trace and store photons” from “query stored photons while rendering the image.”



Two-pass thinking is the key idea: write light first, read light later.

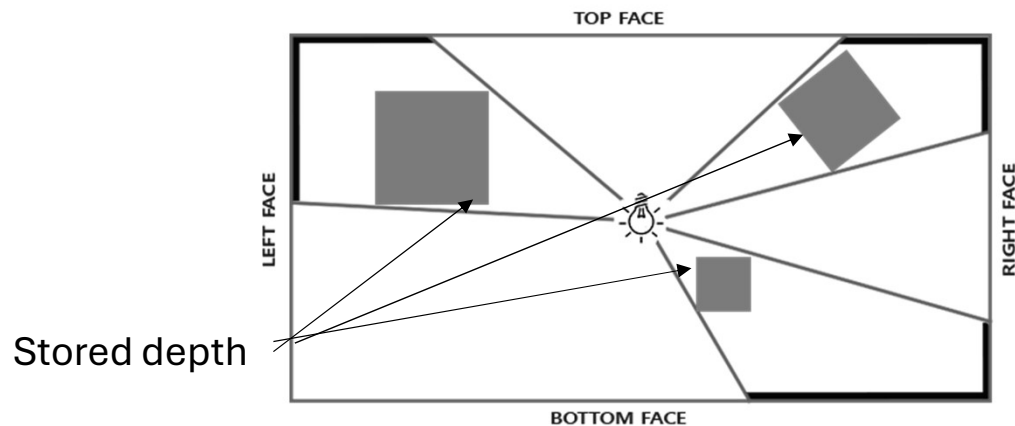
Photon Mapping Workflow (Build 2/2)

- **Pass 1:** emit photons from lights, scatter them through materials, and store selected diffuse hits.
- **Pass 2:** trace eye rays, find the visible surface point, and gather nearby photons to estimate indirect radiance.
- *Direct lighting* is often computed separately with explicit shadow rays, while the *photon map* focuses on *indirect light and caustics*.
- Because the gather is **local**, photon mapping can reuse a large preprocessing effort across many camera views.



Shadow Mapping Idea

- Render the scene once from the light's point of view and store the nearest depth in a shadow map.
- During camera rendering, transform each fragment into light space and compare its depth (relative to the light source) with the stored value.
- If the fragment is farther than the stored depth it's in shadow.
- The method is GPU-friendly because the main data structure is a depth texture, not explicit shadow geometry.



Pass 1

light view
render depth

Store

Depth of all
closest face to
the light

Pass 2

camera view
compare depths
of face to the
stored one

From the light

the shadow map
remembers only the
first visible surface
for each sampled
light direction

At the fragment

if the current point is
farther than the stored
depth, some other
surface blocked the
light

**shadow test = "am I behind the first
surface seen by the light?"**

Shadow Mapping Common Artifacts

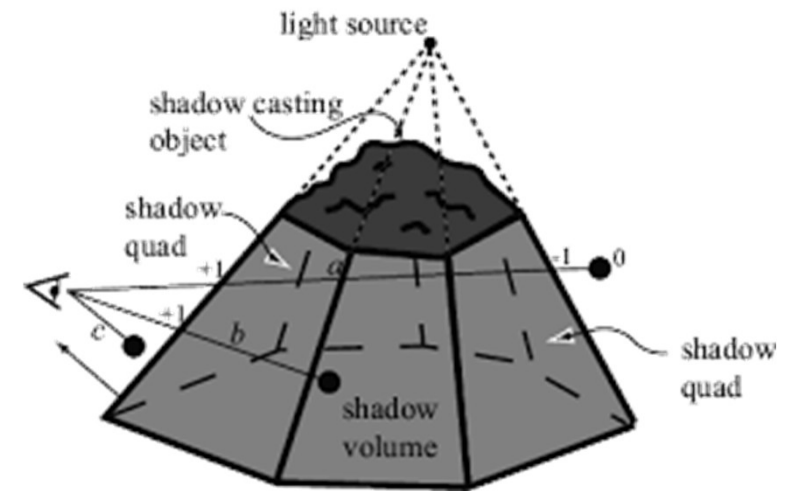
- **Shadow acne** happens when numerical precision makes a lit fragment compare against nearly equal stored depth. Use bias to solve this.
- **Peter-panning** happens when bias is too large and the shadow detaches from the caster.
- **Aliasing (Jagged Edges)**: If the shadow map resolution is too low, the shadows look like they are made of giant Lego bricks.

Peter-panning



Shadow Volumes Idea

- Shadow volumes start from **silhouette edges**: places where one face sees the light and the adjacent face does not.
- Each silhouette edge is **extruded away** from the light to create a closed volume of shadowed space.
- The stencil buffer counts whether the eye ray enters and leaves that volume before hitting a visible surface.
- This gives crisp, resolution-independent hard shadows without storing a depth map.



Unlike shadow mapping, the method stores shadow space as geometry instead of a depth texture.

Shadow Volumes: Strengths and Trade-Offs

- **Strength:** hard-shadow edges are exact with respect to the shadow volume geometry, not shadow-map texels.
- **Strength:** image resolution changes do not change the geometric shadow boundary.
- **Cost:** silhouette extraction and volume rendering become expensive for very complex or animated meshes.
- **Limitation:** the classic approach is best for hard shadows; soft shadows usually need extensions or another method.

Pros

Resolution independent
Crisp hard edges
No shadow texture needed

Cons

Silhouette cost
Stencil complexity
Hard to soften naturally

Decision rule

Need crisp hard edges?

Yes

Closed mesh?

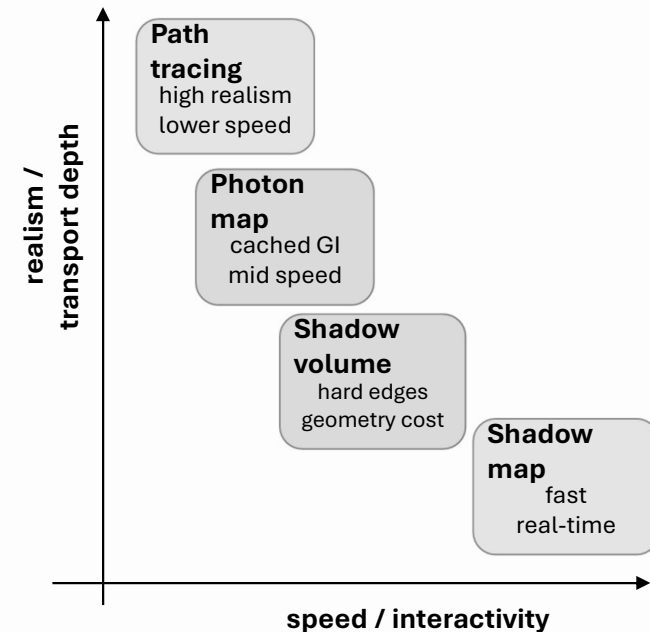
Yes

Many lights / huge geometry?

May be not ideal

Method Comparison and Choice Guide

- **Path tracing:** highest realism and simple concept, but expensive and noisy at low samples.
- **Photon mapping:** good for indirect light reuse and caustics, but requires preprocessing and data structures.
- **Shadow mapping:** fast and standard for real-time rasterization, but depth bias and aliasing require care.
- **Shadow volumes:** crisp hard shadows and no texture aliasing, but more geometry and stencil complexity.



Key Takeaways

- **Path tracing** follows eye paths and is the cleanest general GI model, but it needs enough samples to control noise.
- **Photon mapping** reuses precomputed light particles, which is especially helpful for indirect light reuse and caustics.
- **Shadow mapping** is the default real-time workhorse: easy to deploy, sensitive to depth precision and map resolution.
- **Shadow volumes** trade shadow-map aliasing for geometry and stencil work, producing sharp hard shadows.
- As designers and engineers, you should now be able to explain, choose, and prototype these methods.

Explain

what the method solves

Choose

match it to the task

Prototype

implement a minimal version

Debug

know the typical failure mode

skill gain after Lecture 6

understand light transport
implement a shadow / GI
baseline

compare methods like an
engineer

Homework and Further Reading

- 1) Implement 3 types of shadow techniques (shadow mapping, shadow volume, path tracing) on the lecture 4 practice (a rotating cube with texture & light)
- 2) Create a “background” below the cube, which is large and far enough to capture the shadows.
- 3) Use call back function to switch between 3 types of shadow techniques
- 4) Explain how to modify the existing code to implement shadow (highlight the change in the code)

Deliverable 1 code with highlight and explain (docx)

Deliverable 2 Code file (.cpp)

Submit on Teams